Lab Assignment 10.1

1. Syntax and Logic Errors

   Task:

   Use AI to identify and fix syntax and logic errors in a faulty Python script.

   Sample Input Code:

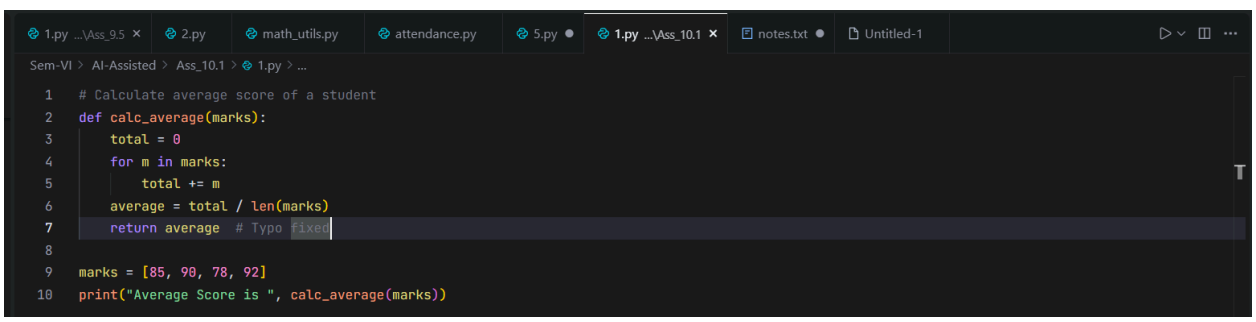
```python
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return avrage  # Typo here

marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)
```

   Expected Output:

   - Corrected and runnable Python code with explanations of the fixes.

Fixed Code:



2. PEP 8 Compliance

   Task:

   Use AI to refactor Python code to follow PEP 8 style guidelines.

   Sample Input Code:

```python
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
```

   Expected Output:

   - Well-formatted PEP 8-compliant Python code.

Well Formatted Code:

```python
def area_of_rect(length: float, breadth: float) -> float:
    """
    Returns the area of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    breadth (float): The breadth of the rectangle.

    Returns:
    float: The area of the rectangle.
    """
    return length*breadth
print(area_of_rect(10,20))
```

3. Readability Enhancement
   Task:
   Use AI to make code more readable without changing its logic.
   Sample Input Code:

```
def c(x,y):
    return x*y/100
a=200
b=15
print(c(a,b))
```

   Expected Output:
   - Python code with descriptive variable names, inline comments, and clear formatting.

Enhanced Readability:

```python
def calculate_percentage(x: float, y: float) -> float:
    """
    Returns the result of multiplying x and y, then dividing by 100.

    Parameters:
    x (float): The first number to be multiplied.
    y (float): The second number to be multiplied.

    Returns:
    float: The result of (x * y) / 100.
    """
    return x*y/100
a = 200
b = 15
print(calculate_percentage(a,b))
```

4. Refactoring for Maintainability
   Task:
   Use AI to break repetitive or long code into reusable functions.
   Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Modular Reusable Function:

```python
# Use AI to break repetitive or long code into reusable functions
def welcome_student(name: str) -> None:
    """
    Prints a welcome message for the given student name.

    Parameters:
    name (str): The name of the student.
    """
    print("Welcome", name)

students = ["Alice", "Bob", "Charlie"]
for student in students:
    welcome_student(student)
```

5. Performance Optimization

   Task:

   Use AI to make the code run faster.

   Sample Input Code:

```python
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

   Expected Output:

   Optimized code using list comprehensions or vectorized operations.

Optimized Code:

```python
# using list comprehension
nums = [i for i in range(1,1000000)]
squares = [n**2 for n in nums]
print(len(squares))

# using vectorized operations
import numpy as np
nums = np.array(nums)
squares = nums**2
print(len(squares))
```

6. Complexity Reduction

   Task:

   Use AI to simplify overly complex logic.

   Sample Input Code:

```python
def grade(score):
    if score >= 90:
        return "A"
    else:
```

```
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Simplified code

1.py × | 2.py ...\Ass_9.5 | math_utils.py | attendance.py | 5.py ...\Ass_9.5 ● | 2.py ...\Ass_10.1 ● | 3.py ● | 4.PY ● | 5.py ...\Ass_10.1 ● | 6.py ●

Sem-VI > AI-Assisted > Ass_10.1 > 6.py > ...

```
# Simplified using elif statements
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

# Using dictionary lookup
def grade(score):
    grades = {
        (90, float("inf")): "A",
        (80, 90): "B",
        (70, 80): "C",
        (60, 70): "D",
        (0, 60): "F"
    }
    for (low, high), grade in grades.items():
        if low <= score < high:
            return grade
```